Article Spinner and Cipher Decryption

What is Article Spinning

- **Definition**: The process of rewriting articles to make them appear unique to search engines.
- **Purpose**: Helps website owners by reducing content creation efforts, saving time and costs, and boosting SEO through link-building.
- **Objective** of this topic: Not to create an article spinning tool, but to explore probabilistic models (QQ).

人工智慧(AI)正在迅速改變我們的生活方式。從語音助手到自動駕駛技術，AI 已經滲透到我們日常生活的各個領域。許多企業利用 AI 來提高效率，改善客戶體驗，甚至進行預測分析。隨著技術的不斷發展，AI 的應用範圍將會越來越廣泛。

→

人工智慧(AI)正以驚人的速度改變著現代社會。從智慧語音助手到自駕技術，AI 的影響無處不在。企業紛紛採用 AI 來增強作業效率，改善使用者體驗，並預測市場趨勢。隨著技術的飛速發展，AI 的應用將變得更加普遍。

Example

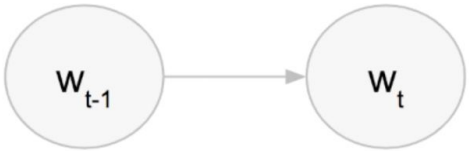
假設你經營一個有關健身的網站，你希望這個網站在 Google 上的排名更高。你可以：

1. 撰寫一篇文章，例如「10 個提升肌肉增長的秘訣」。
2. 使用 Article Spinning 來創造多個不同版本的這篇文章，內容大致相同但用詞不同。
3. 發佈這些文章到不同的部落格或論壇，並在裡面加上回到你網站的連結。
4. 搜尋引擎會看到你的網站被很多不同的網站連結，認為你的內容有價值，於是提升排名。

N-Gram Markov Models

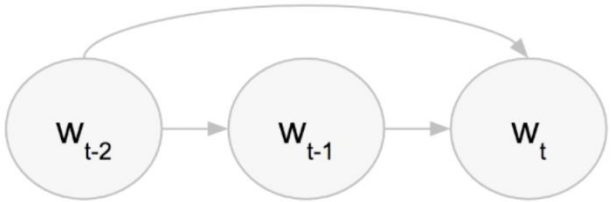
- First-order Markov model

$$p(w_t \mid w_{t-1}) = \frac{count(w_{t-1} \rightarrow w_t)}{count(w_{t-1})}$$



- Secod-order Markov model

$$p(w_t \mid w_{t-1}, w_{t-2}) = \frac{count(w_{t-2} \rightarrow w_{t-1} \rightarrow w_t)}{count(w_{t-2} \rightarrow w_{t-1})}$$



Predicting the Middle Word

- For generating poetry, we take the two previous words and use them to generate the next word
- For article spinning, we don't want to generate text from start to end
- We want to replace text, such that it makes sense in the context of what came before and what comes after
- Idea: use a trigram to predict the middle word from surrounding words

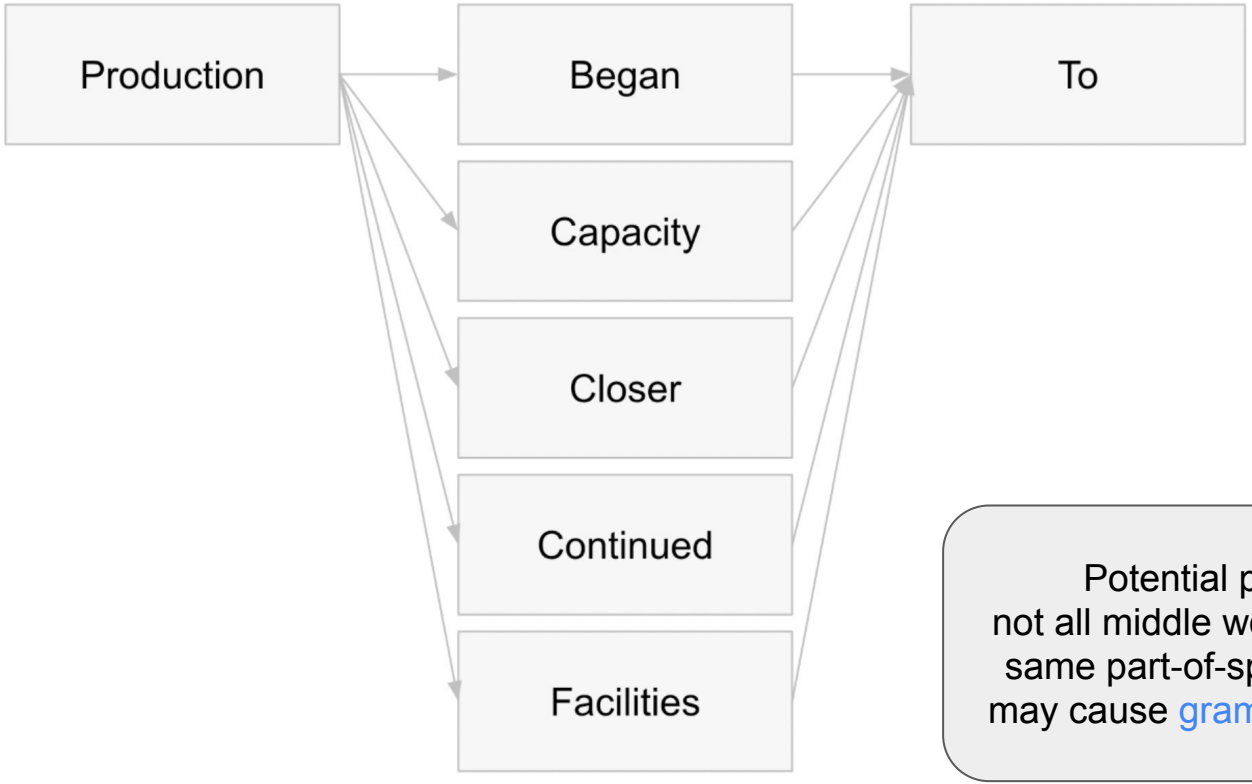
$$p(w_t \mid w_{t-1}, w_{t+1})$$

Estimating the Middle Word Distribution

- Maximum likelyhood estimation as usual

$$p(w_t \mid w_{t-1}, w_{t+1}) = \frac{count(w_{t-1} \rightarrow w_t \rightarrow w_{t+1})}{count(w_{t-1} \rightarrow ANY \rightarrow w_{t+1})}$$

Does it Work?



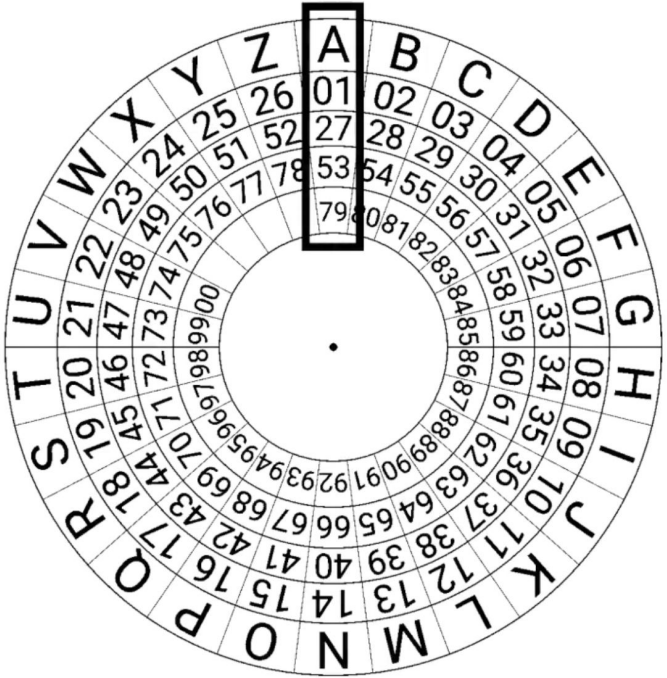
Potential problem:
not all middle words have the same part-of-speech, which may cause grammatical issue

Exercise Prompt

- Dataset: BBC News Data
- We’ll use business articles only
- Feel free to use all articles, or even a different dataset (e.g. wikipedia)
- Build the model (VxVxV matrix or Python dictionary)

Cipher Decryption

- Decrypt an encoded message
- Applies multiple important topics:
  - Probabilistic language modeling
  - Genetic algorithms
- Applications: warfare, espionage



Section Outline

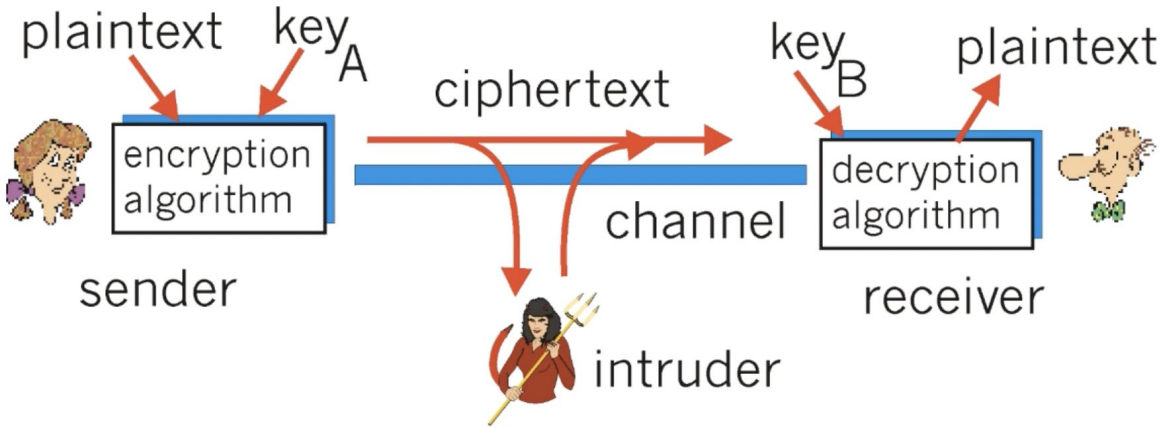
- What is a cipher?
  - Encode and decode a message
- Language modeling
  - What is the probability of a sentence?
- Genetic algorithm / evolutionary algorithm
  - Optimization based on biological evolution
- Our decoded message should have the highest likelihood if the model is trained on the English language
- A message not in English should have a smaller likelihood

Substitution Cipher

- We encode the message by substituting each letter with a different letter

High-Level Picture

- Sender sends a message to the receiver
- A spy / intruder has intercepted the message, but it is encrypted
- The receiver decrypts the message
- Both sender/receiver can encrypt/decrypt using a key (dictionary that maps plaintext to ciphertext)



Example (Sender)

- Plaintext message: “I LIKE CATS”
- Substitution:
  - I→Y, L→W, I→Y, K→R, E→N, C→J, A→L, T→O, S→B
- Ciphertext message: “Y WYRN JLOB”

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
L	X	J	S	N	H	I	Z	Y	P	R	W	C	V	G	E	D	A	B	O	T	K	M	Q	F	U

Example (Receiver)

- Ciphertext message: “Y WYRN JLOB”
- Substitution:
  - Y→I, W→L, Y→I, R→K, N→E, J→C, L→A, O→T, B→S
- Plaintext message: “I LIKE CATS”

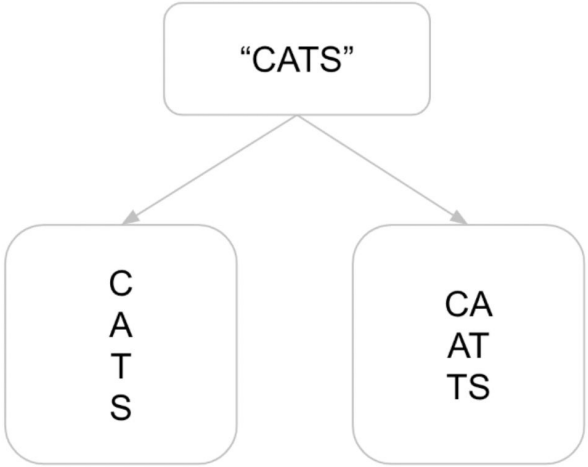
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
L	X	J	S	N	H	I	Z	Y	P	R	W	C	V	G	E	D	A	B	O	T	K	M	Q	F	U

Language Modeling

- Intuition: build a model that assigns high probability to real words / sentences, and low probability to unreal words / sentences
- MyModel(“I LIKE CATS”) → big number
- MyModel(“Y WYRN JLOB”) → small number
- So, if we decrypt the message correctly, our model should return a high probability (and a low probability if decrypt incorrectly)

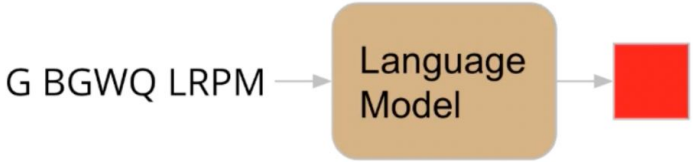
N-Gram Models in Cipher Decryption

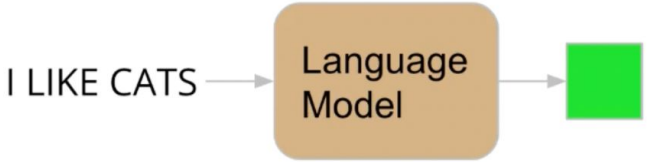
- N-gram: A sequence of N tokens (letters in this case)
- Tokens usually refer to words, but here we use individual letters
- Why use N-grams for cipher decryption?
  - Unigrams (1-gram): Helps identify likely letter frequencies (e.g., "E" is most common in English)
  - Bigrams (2-gram): Captures common letter sequences to improve accuracy
    - ✅ Common bigrams: TH, HE, ER, IN, AN
    - ❌ Rare bigrams: ZX, QP, JK, XZ, VQ
- We use unigram & bigram models to evaluate decryption quality



How to Use the Model

- First, we find the individual bigram / unigram probabilities using a large text corpus (e.g. Moby Dick, wikipedia)
- Then, we can calculate the probability of any sentence of word
- Suppose I translate the message incorrectly: “G BGWQ LRPM”
  - Model should return low probability
- If I translate the message correctly: “I LIKE CATS”
  - Model should return high probability
- I want to find the translation / decryption that yields the maximum likelihood





How to Use the Model

- First, consider a naive approach

```
decryption_maps = get_all_possible_decryption_maps()
best_log_likelihood = -inf
best_message = None

for map in decryption_maps:
  message = decode(encrypted_message, map)
  log_likelihood = calculate_log_likelihood(message)
  if log_likelihood > best_log_likelihood:
    best_log_likelihood = log_likelihood
    best_message = message

print("Final message:", message)
```

How to Use the Model

- First, consider a naive approach

Problem! How do we find all feasible decryption maps?

```
decryption_maps = get_all_possible_decryption_maps()
best_log_likelihood = -inf
best_message = None

for map in decryption_maps:
  message = decode(encrypted_message, map)
  log_likelihood = calculate_log_likelihood(message)
  if log_likelihood > best_log_likelihood:
    best_log_likelihood = log_likelihood
    best_message = message

print("Final message:", message)
```

Infeasible!

- We have 26 possible letters

26	25	...	2	1

- Number of probabilities = 26 x 25 x 24 x … x 2 x 1 = 26! ≅ 4×10^{26}
- Even if each possibility takes only 1 nanosecond to check, you still have to wait **12.7 billion years**

Genetic Algorithms

- An optimization approach, similar to **gradient descent** in ML
- Parent passes on DNA to children (offspring)
- E.g. if you have black hair, your children will have black hair
- Child may not be an exact copy of the parent

Genetic Algorithms

- The offspring takes DNA only from the parent, therefore, is a copy
- Mistakes can happen
- DNA is a string (like in computer programming)
- But instead of 26 letters (A-Z) we only have 4: A, T, C, G

Types of Mistakes (Mutation)

(1) Substitution

(2) Insertion

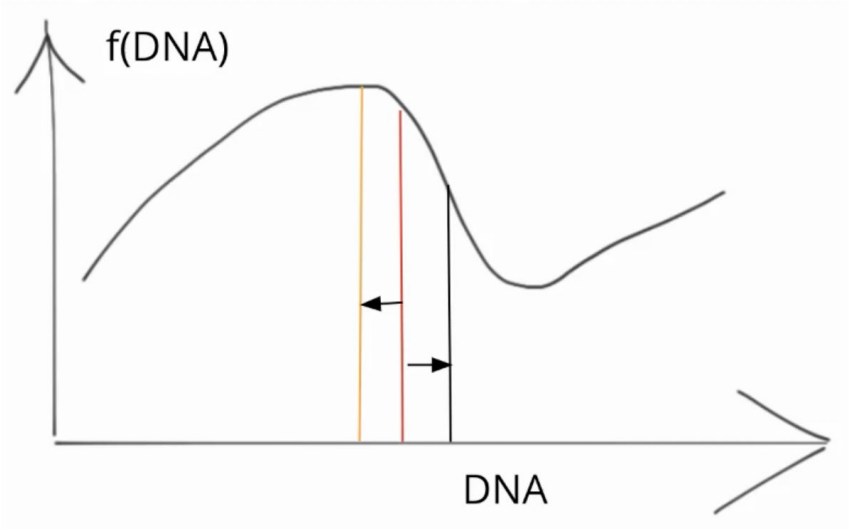
(3) Deletion

Genetic Algorithms

- Overtime, DNA mutations build up - those with the highest “fitness” will persevere
- Mutations can be good or bad
- Bad mutations = genetic diseases
- Objectively, “unfit” genes will not propagate as well to next generation
- “Fit” genes procreate more, “unfit” gene less, until we end with the “most fit” genes

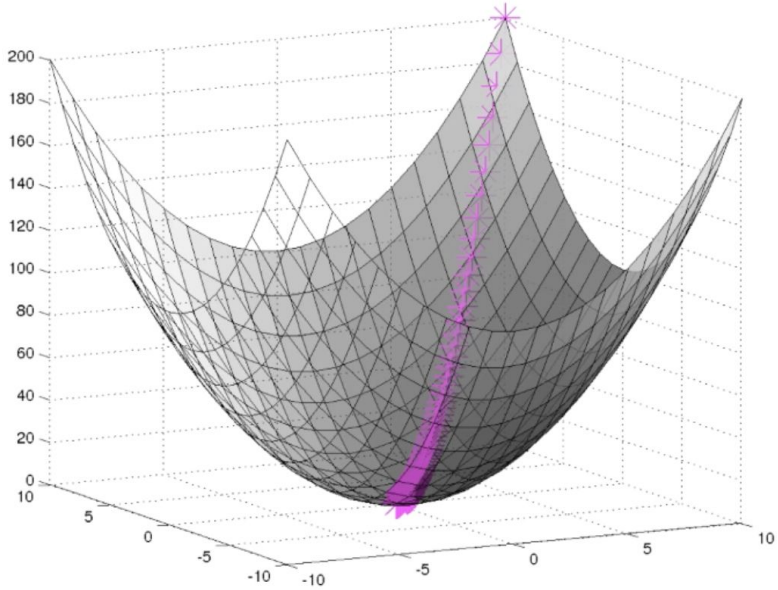
Numerical Optimization

- Think of it as a numerical optimization (maximization, max*f(x)*)
- Fitness is a function (f) of DNA
- At each generation, we create multiple offspring, and check their fitness, and keep only the most fit



Numerical Optimization

- Optimization is a common task in ML
- Why not use gradient descent?
- Gradient = derivative of f(x)
- Objective = log-likelihood
- Parameters = map from coded letter to plaintext letter
- It’s not differentiable!



Back to Genetic Algorithms

- How do we represent our model parameters as a “DNA string”?
- Unlike DNA, it’s not just ATCG
- Our model is just a mapping / dictionary
- The inputs (keys) are the alphabet in alphabetical order
- We only need to care about the values (i.e. a string like: LXJSN…)
- Contraints: every letter must appear once, no repetition

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
L	X	J	S	N	H	I	Z	Y	P	R	W	C	V	G	E	D	A	B	O	T	K	M	Q	F	U

Genetic Algorithms

- What is our function “f”?
- Input should be a DNA string (representing the values of map)
- Output should be a fitness value (log-likelihood)

```
def f(dna_string):
  map = convert_dna_string_to_map(dna_string)
  message = decode(ciphertext, map)
  log_likelihood = model(message)
  return log_likelihood
```

Convert DNA String to Map

- Given a DNA string, write a function that returns the corresponding dictionary mapping
- E.g.
  Input = “LXJSN…”
  Output = {“A”: “L”, “B”: “X”, “C”: “J”, “D”: “S”, “E”: “N”, …}

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
L	X	J	S	N	H	I	Z	Y	P	R	W	C	V	G	E	D	A	B	O	T	K	M	Q	F	U

How to Mutate a DNA String?

- So far:
  - We understand how to represent DNA in code
  - We understand how genetic algorithm works (at a high-level)
- Before pseudocode, we first have to understand what it means to mutate our DNA
- In biology, ATCGs can be mutated by insertion, deletion, substitution
- Our “DNA” is the 26 letters of the alphabet
  - Insertion / deletion: no longer 26 letters → not allowed
  - Substitution: letters won’t be unique → not allowed

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
L	X	J	S	N	H	I	Z	Y	P	R	W	C	V	G	E	D	A	B	O	T	K	M	Q	F	U

Mutation

- In order to meet the constraints of our DNA, we can use swapping
- Mutation rate should be small, so each child will only swap 1 pair of letters (i.e. switch 2 letters)
- Total number of letters is still 26
- Letters are still unique and each letter appears just once

A	B	C	D

A	C	B	D

Genetic Algorithm

- First, a basic approach (not the final version)

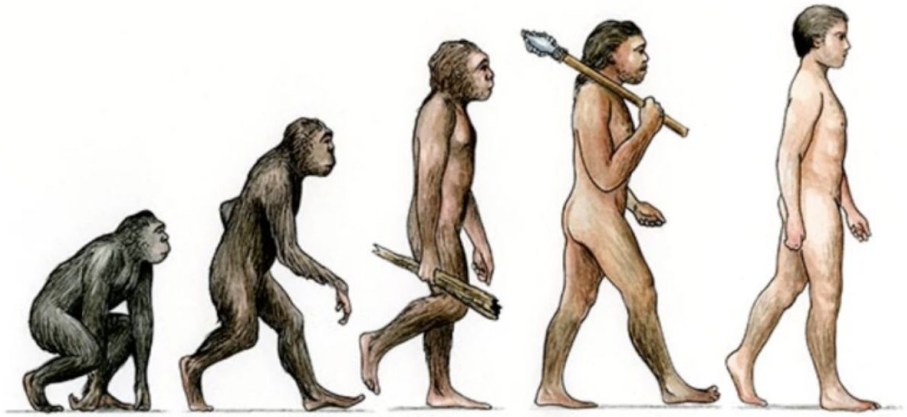
```
DNA = get_random_dna()
fitness = f(DNA)

for _ in range(num_epochs):
  DNA_new = randomly_switch(DNA)
  fitness_new = f(DNA_new)

  if fitness_new > fitness:
    DNA = DNA_new
    fitness = fitness_new
```

Genetic Algorithm Improvement

- Too easy to get stuck in a local optimal
- Mathematically, w'd like to search many locations / directions at once
- Each generation will consist of multiple DNA strings
- Each DNA string will have multiple offspring
- At each generation, not every individual will survive to procreate - only the fittest



Genetic Algorithm Pseudocode

```
DNA_pool = get_many_random_dna(20)

for i in range(num_epochs):
  if i > 0:
    # 3 offspring per parent
    DNA_pool = create_offspring(DNA_pool, 3)

  scores = [f(DNA) for DNA in DNA_pool]

  # sort DNA by score
  # keep only top 5
  DNA_pool = sorted_DNA[:5]

  # 15 children + 5 parents = 20
```

Exercise Details

- Generate a random substitution cipher
- Read in Moby Dick, create a character-level language model
  - If alphabet has V letters, then there are V unigrams and VxV bigrams
  - f(message) → log likelihood
- Encoding and decoding functions
  - Encoding function used to encode message using the true cipher
  - Decoding function used to test guesses
- Genetic algorithm